UNIT – V: Learning

What is learning
Learning by Taking Advice
Learning in Problem-solving
Learning from example: induction, Explanation-based learning

What is Learning?

- Herbert Simon defined learning as changes that enable a system to perform tasks more efficiently over time.
- It focuses on self-improvement in AI, where systems improve performance or expand their knowledge base.

Learning Tasks for an Agent:

1. **Predicting or deciding the outcome of an action:**

This task involves the agent estimating what will happen as a result of taking a particular action. For example, if the agent is playing a game, it needs to predict whether making a certain move will lead to a favorable or unfavorable outcome, based on its current knowledge.

2. **Evaluating states based on their value:**

In this task, the agent assesses the "value" or "goodness" of different states it encounters. A state is a snapshot of the environment, and the agent evaluates how beneficial or desirable that state is, often in relation to achieving a goal. For instance, in a chess game, a state where an agent has captured the opponent's queen might be considered more valuable than a state where it's losing pieces.

3. **Keeping track of relevant perceptions:**

Here, the agent is responsible for remembering and updating important information from its surroundings (perceptions). These perceptions are the data or inputs the agent receives from its environment. For instance, in a self-driving car, the agent must track information like the positions of nearby vehicles, traffic signals, and road conditions to make informed decisions.

Types of Learning:

1. **Rote Learning (Memorization):** Memorizing without understanding the logic.

   Rote learning is based on memorization, focusing on recalling information without understanding the underlying complexities.The learner memorizes facts or sequences through repetition, making it easier to recall the stored knowledge.

   Example: Learning a poem or song by repeatedly reciting it, without understanding the meaning behind the words.

2. **Passive Learning (Instructions):** Learning through direct instruction from a teacher or expert.

3. **Analogy (Experience):** Learning from past experiences and applying them to new situations.
4. **Inductive Learning (Experience):** Inductive learning is a type of machine learning where the agent learns general rules or patterns from specific observations. Instead of being explicitly programmed with rules, the agent uses the data it encounters to infer general principles or models.

   For example, given a set of labeled examples (such as pictures of cats and dogs), an inductive learning algorithm would learn to generalize from these examples to recognize new, unseen images of cats and dogs. The key is that the agent "induces" general knowledge from specific instances rather than simply memorizing them.

5. **Deductive Learning:** Deriving new facts from existing ones.

Learning by Taking Advice means that a system (like a computer or AI) learns by following guidance or suggestions from an expert (like a programmer or human).

Concept of Learning by Advice:

- **Advice** is given by a teacher (like a programmer) to a student (the computer).
- The computer uses this advice to perform tasks it couldn't do before, but it must first translate the advice into actions it can understand and follow.

Advice Processing in AI:

- Just like people interpret advice, a computer must turn abstract advice into specific actions.
- For example, in chess, the advice "control the center of the board" needs to be turned into specific moves the computer can play.

FOO Example:FOO is a program that can take advice about playing strategies for the card game Hearts.

- The advice might be abstract, like “play high cards when safe,” and FOO must learn how to apply this in real games.
- If FOO makes mistakes, the human can give more specific advice to help it improve.

Another example: FOO uses advice to create executable procedures that can be run by the system, such as writing in a programming language like LISP.

The abstract advice is converted into detailed rules that the system can use to make decisions or Special tools help convert expert advice into specific rules that the system can understand and use.

In short, learning by advice helps an AI system improve by translating general guidance into actionable steps it can follow.

Learning in problem solving refers to the process where an AI system improves its ability to solve problems by gaining experience, adjusting its strategies, and refining its knowledge, all without requiring a human teacher.

Instead of acquiring new information, it reorganizes existing data, optimizes its approach, and applies lessons from past experiences to similar problems in the future.

Key Characteristics:

- **Self-Improvement**: The system enhances its problem-solving capabilities by analyzing past outcomes.
- **Generalization**: It learns to apply solutions from one problem to others that are similar.
- **Efficiency**: The goal is to solve similar problems more efficiently over time.

Types of Learning in Problem Solving:

1. Learning by Parameter Adjustment:

- **Concept:** AI systems use an evaluation function to make decisions, often based on multiple factors. By adjusting the importance (weights) of these factors, the system can improve its decision-making.
- **Example:** In a chess program, factors like material advantage (pieces on the board) and mobility (available moves) are considered. Initially, both factors might have equal importance. Over time, the program learns that material advantage is more important early in the game and adjusts the weights accordingly, improving its performance.

2. Learning by Macro-operators:

- **Concept:** A macro-operator is a sequence of actions treated as a single operation, making problem-solving faster by reducing repetitive steps.
- **Example:** In solving a Rubik's Cube, rather than solving each face one by one, the system learns a shortcut (macro-operator) that solves multiple steps at once, speeding up the solution process.

3. Learning by Chunking:

- **Concept:** Chunking is the process of combining smaller pieces of information into a single "chunk" for easier use and faster recall.
- **Example:** In solving algebra problems, a system might chunk together the rules for adding fractions and simplifying, so when it encounters a new problem, it applies the chunked knowledge in one step, saving time.

These methods help AI systems learn to solve problems more efficiently by improving how they evaluate, process, and organize information.

Learning from Example: Induction is a process where a system learns general rules or concepts from specific examples. This is a type of **supervised learning**, where the system is given labeled examples and must learn to classify or predict new instances based on those examples.

Key Characteristics of Inductive Learning:

1. **Generalization:**
    - The system looks at specific examples and tries to find a general rule that can apply to all similar cases.
2. **Supervised Learning:**
    - The system is trained on examples that have both inputs (data) and outputs (labels or classifications). It uses these to learn.
3. **Classification and Concept Formation:**
    - The system forms concepts (e.g., categories or classes) to distinguish between different types based on the provided examples.

Techniques for Inductive Learning:

1. **Winston's Learning Program:**
    - The system uses positive and negative examples to learn structural concepts.
    - **Example:** Learning the concept of "arch" by recognizing key features (like a top block supported by two side blocks) in the examples.
2. **Version Spaces:**
    - The system maintains a space of possible hypotheses and gradually refines it as more examples are introduced.
    - **Example:** Given different shapes, the system narrows down the definition of a "triangle" by eliminating hypotheses that don't fit the examples.
3. **Decision Trees:**
    - A decision tree model is built for classification, where decisions are made based on different features.
    - **Example:** Classifying fruits like "apple" or "orange" based on attributes like color, size, and taste.

Example of Inductive Learning:

Consider a system learning to classify animals:

- **Input Examples:**
    - A dog has four legs, fur, and barks (labeled as "Dog").
    - A cat has four legs, fur, and meows (labeled as "Cat").
- **Generalization:**
    - The system learns rules such as:
        - "If it has four legs, fur, and barks, it's a Dog."
        - "If it has four legs, fur, and meows, it's a Cat."

As more examples are added, the system refines these rules to improve its accuracy in classification.

Explanation-Based Learning (EBL) is a type of machine learning where a system learns by analyzing a specific example in detail and generalizing from it. EBL focuses on understanding **why** a solution works, not just **how** it works, using prior knowledge to create general principles that apply to similar problems.

Key Concepts in EBL:

1. **Training Example:**The system starts with a specific problem or example it needs to solve.
2. **Domain Knowledge:**The system uses pre-existing knowledge (rules or facts) about the problem area to explain why the solution works.
3. **Explanation Generation:**The system explains how the solution works, reasoning through the steps using the knowledge available.
4. **Generalization:**After understanding the example, the system extracts general rules that can be used for future, similar problems.

Example of EBL:

- **Problem:** A system is given a shape with three sides and asked if it's a triangle.
- **Domain Knowledge:** The system knows that a triangle is a polygon with three sides.
- **Explanation:** The system checks that the shape has three sides, matching the definition of a triangle.
- **Generalization:** The system creates the rule: "Any polygon with three sides is a triangle." It can now use this rule for future shapes.

Steps in EBL:

1. **Input an Example:** Start with a specific case
2. **Explain the Solution:** Use prior knowledge to explain why the solution works.
3. **Derive a General Rule:** Extract a general rule from the explanation.
4. **Apply the Rule:** Store the rule to solve similar problems in the future.

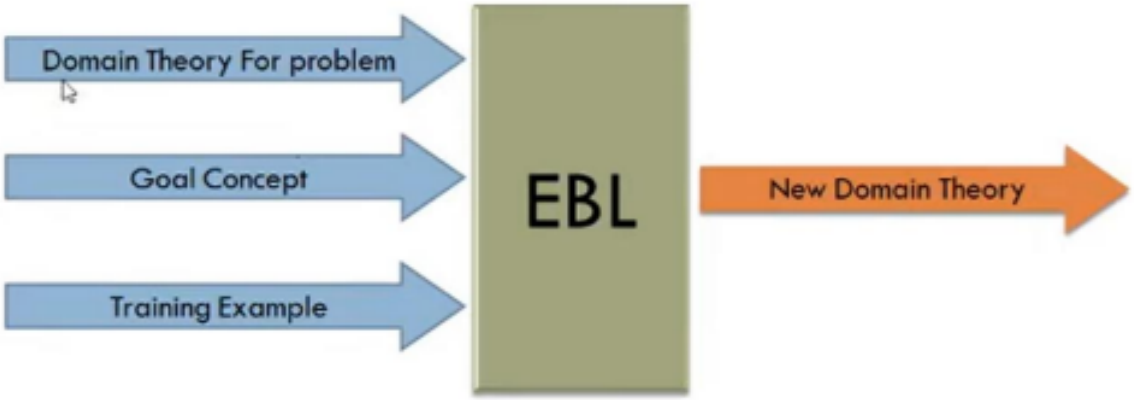
Benefits of EBL:

- **Efficiency:** Learn from a single example, making future problem-solving faster.
- **Focus on Relevant Information:** Only the important principles are learned, ignoring irrelevant details.
- **Improves Problem-Solving:** By understanding the "why," the system avoids mistakes and improves decision-making.

Applications of EBL:

- **Game Playing:** Chess engines use EBL to learn strategic rules.
- **Natural Language Processing (NLP):** Learning grammar rules from example sentences.
- **Medical Diagnosis:** Generalizing diagnostic rules from specific patient cases.

Explanation based learning



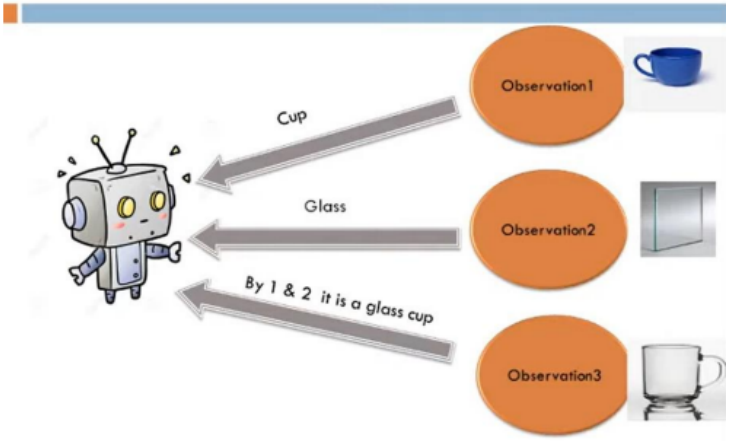
Domain Theory: Set of rules and facts that are used to explain how the examples is a case of the goal concept.

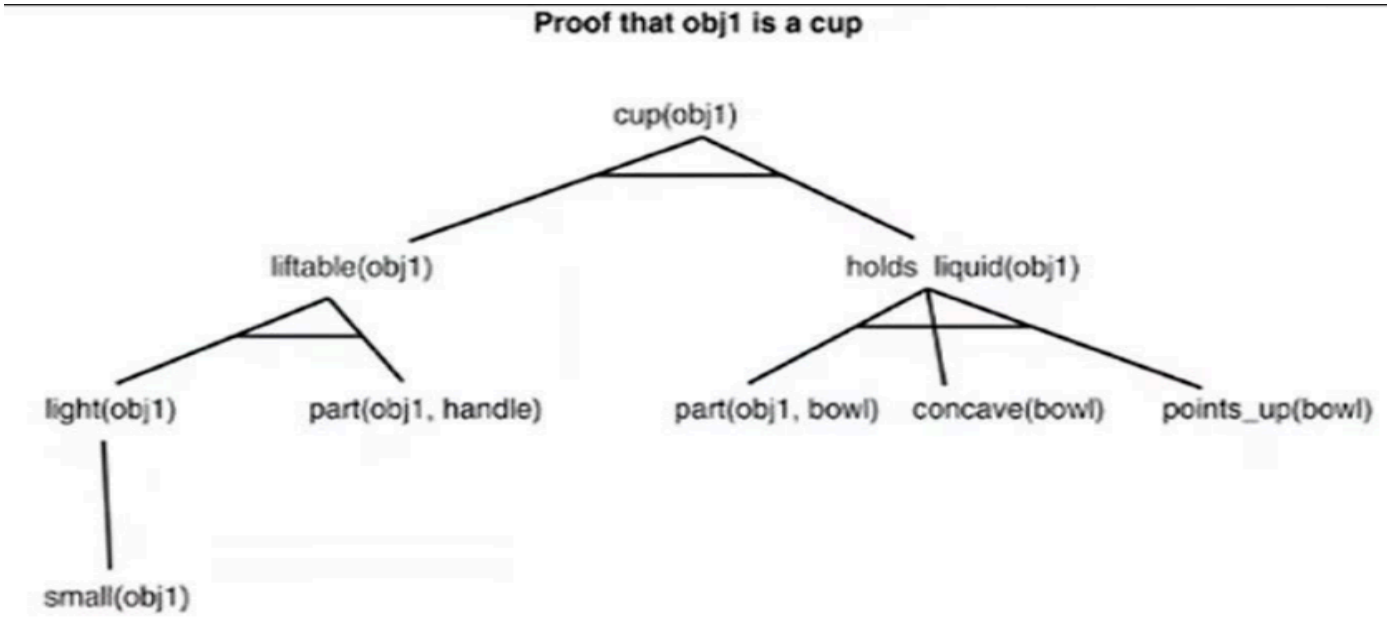
Explanation based learning

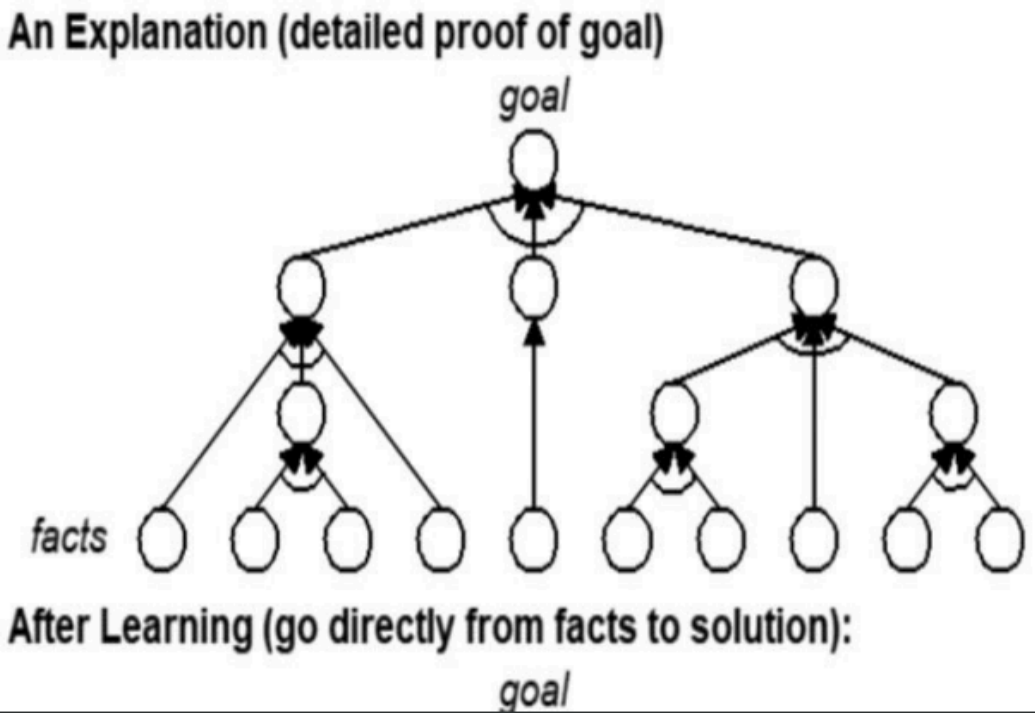
Example

- target concept : a rule used to infer whether an object is a cup
  - premise(X) -> cup(X)
- domain theory
  - liftable(X) ^ holds_liquid(X) -> cup(X)
  - part(Z, W) ^ concave(W) ^ points_up(W) -> holds_liquid(Z)
  - light(Y) ^ part(Y, handle) -> liftable(Y)
  - small(A) -> light(A) . made_of(A, feathers) -> light(A)
- training example : an instance of the goal concept
  - cup(obj1) , small(obj1), part(obj1, handle), owns(bob, obj1), part(obj1, bottom), part(obj1, bowl), points_up(bowl), concave(bowl), color(obj1, red)

Explanation based learning







PART 2 EXPERT SYSTEM

Representing and using Domain Knowledge

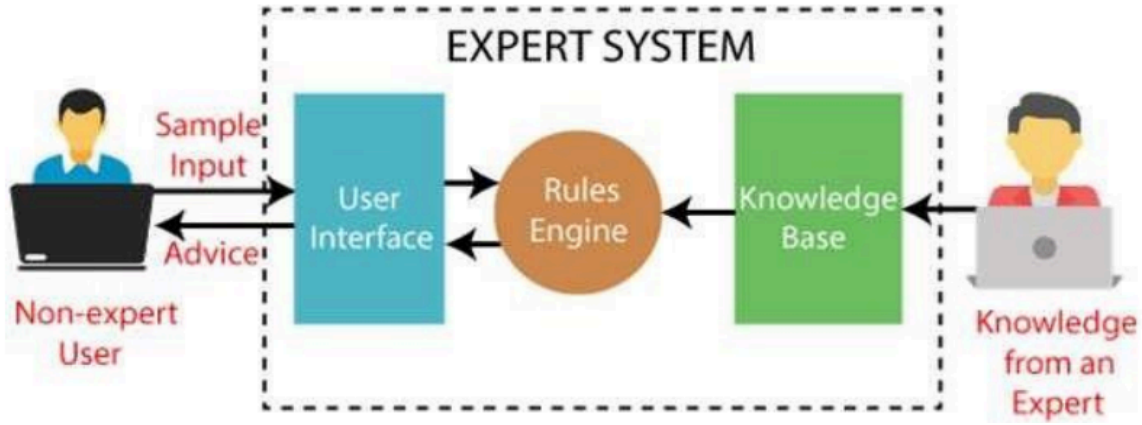
Reasoning with knowledge

Expert System Shells-examples

Knowledge acquisition skills-examples

Expert System (ES)

- **Definition**: An expert system is a computer program designed to solve complex problems and make decisions similar to a human expert.
- **Function**: It extracts knowledge from its knowledge base and applies reasoning and inference rules to respond to user queries.
- **AI Integration**: Part of Artificial Intelligence (AI), the first ES was developed in 1970, marking a significant step in AI development.
- **Performance**: Its performance depends on the knowledge stored in the knowledge base. The more knowledge it has, the better it performs.
- **Application**: It uses facts and heuristics for decision-making, much like human experts.
- **Example**: A common example is spelling error suggestions in Google's search box.
- An expert system assists human experts in making complex decisions, rather than replacing them.



Popular Examples of Expert Systems:

1. **DENDRAL**: Used in organic chemistry for detecting unknown molecules based on mass spectra and chemical knowledge.
2. **MYCIN**: Early expert system used for diagnosing bacterial infections and recommending antibiotics, particularly for blood diseases like bacteraemia and meningitis.
3. **PXDES**: Determines lung cancer type and severity by analyzing images resembling body shadows.
4. **CaDeT**: A diagnostic system for detecting cancer at early stages.

Characteristics of Expert Systems:

1. **High Performance**: Solves complex problems efficiently and accurately in specific domains.
2. **Understandable**: Interacts in a way that users can easily understand, providing input and output in human language.
3. **Reliable**: Produces accurate and dependable results.
4. **Highly Responsive**: Delivers results quickly, even for complex queries.

Components of an Expert System:

1. **User Interface**:
    - Allows users to interact with the system by inputting queries in a readable format.
    - Displays the output from the system after processing by the inference engine.
2. **Inference Engine**:
    - Applies reasoning to extract knowledge from the knowledge base and draw conclusions.
    - **Types**:
        - **Deterministic Inference Engine:** Makes conclusions based on facts and rules, assuming the conclusions are true.
        - **Probabilistic Inference Engine:** Deals with uncertainty and makes conclusions based on probabilities.
    - **Modes of Inference**:
        - **Forward Chaining:** Starts with known facts and uses rules to infer new conclusions (e.g., predicting trends).
        - **Backward Chaining:** Starts with a goal and works backward to verify facts (e.g., diagnosing a disease).
3. **Knowledge Base**:
    - A repository that stores the expert system's knowledge, including facts and rules.
    - **Components**:
        - **Factual Knowledge:** Verified facts necessary for decision-making.
        - **Heuristic Knowledge:** Practical insights and rules of thumb used to make decisions.
        - **Knowledge Representation:** How knowledge is stored, usually with rules like If-Else statements.
        - **Knowledge Acquisition:** The process of gathering and organizing knowledge from experts.
4. **Participants in Expert System Development**:
    - **Expert:** Provides domain-specific knowledge.
    - **Knowledge Engineer:** Translates the expert's knowledge into a format usable by the system.
    - **End-User:** Interacts with the system to get solutions or advice.

Limitations of Expert Systems:

- **Incorrect Knowledge:** If the knowledge base is wrong, the system will give incorrect answers.
- **Lack of Creativity:** Can't generate creative solutions like humans.
- **High Costs:** Development and maintenance can be expensive.

Applications of Expert Systems:

- **Designing & Manufacturing:** Used for designing products like camera lenses,cars
- **Knowledge Domain:** In advisory roles, such as tax advice.
- **Finance:** Detects fraud, assesses risk, and gives loan advice.
- Used in medical diagnoses and troubleshooting technical devices.

Representing and Using Domain Knowledge in Expert Systems Domain knowledge is the specific knowledge required to solve problems in a particular field. Effectively representing and using this knowledge is essential for building expert systems, as it helps the system reason, make decisions, and provide solutions.

1. Knowledge Representation is about defining and storing domain knowledge in a way the system can use to make inferences and solve problems. Common methods include:

- **Rules (If-Then):** Knowledge is represented as rules that define relationships between facts. For example:
    - "If the customer is older than 60, recommend health products."
    - "If a customer buys a smartphone, offer accessories."
- **Frames:** Represent concepts as objects with attributes and values. For example, a "Product" frame might include "Price," "Category," and "Stock Level."
- **Semantic Networks:** Graphical structures where concepts are nodes and relationships are links. Used to show hierarchical relationships, like "Car" being a type of "Vehicle."
- **Ontologies:** Define types, properties, and relationships within a domain, enabling complex reasoning tasks.

2. Using Domain Knowledge Once knowledge is represented, expert systems use it for problem-solving, decision-making, and diagnostics. Methods include:

- **Inference Engines:** Mechanisms that apply rules to derive new knowledge or make decisions.
    - **Forward Chaining:** Starts with known facts and applies rules to infer new facts.
    - **Backward Chaining:** Starts with a goal and works backward to check if facts support it.
- **Case-Based Reasoning (CBR):** Solves new problems by comparing them to past cases and adapting the solution.
- **Heuristic Search:** Uses experience to make educated guesses about the best solution when an exhaustive search is impractical.
- **Decision Trees:** Used for classification or decision-making by navigating through possible outcomes based on conditions.

3. Challenges in Using Domain Knowledge

- **Knowledge Acquisition:** Extracting expert knowledge and converting it into a usable form for the system is complex and time-consuming.
- **Incomplete or Ambiguous Knowledge:** Sometimes knowledge is incomplete or unclear, making reasoning difficult.
- **Dynamic Domains:** In fields where knowledge is constantly changing (e.g., medicine or finance), the system needs regular updates to stay accurate.

Reasoning with Knowledge in AI: In AI, reasoning helps machines analyze data, make decisions, and solve problems, simulating human thinking using techniques like machine learning, deep learning, and natural language processing.AI systems use reasoning to simulate human thought processes. They apply logical rules, draw conclusions from known facts, and make predictions or decisions.

Examples of Reasoning in AI:

- **Siri:** Uses reasoning to understand voice commands and respond appropriately, such as setting reminders or suggesting locations.
- **WolframAlpha:** Uses reasoning to solve complex problems, like mathematical calculations, by analyzing data and applying logic.

Types of Reasoning in AI:

1. **Deductive Reasoning:**
    - **Description:** A top-down approach that starts with general facts and uses them to deduce specific conclusions.
    - **Example:** "All humans are mortal. Socrates is a human. Therefore, Socrates is mortal."
2. **Inductive Reasoning:**
    - **Description:** A bottom-up approach that starts with specific observations and generalizes them into broader conclusions.
    - **Example:** Observing that all swans are white and concluding that all swans are white.
3. **Abductive Reasoning:**
    - **Description:** Finds the most likely explanation for an observation, often with incomplete information.
    - **Example:** Seeing smoke and concluding there is a fire, even though there might be other causes.
4. **Commonsense Reasoning:**
    - **Description:** AI's ability to simulate human-like decision-making based on experience and heuristics.
    - **Example:** Understanding that if someone holds an umbrella, it’s probably raining.
5. **Monotonic Reasoning:**
    - **Description:** Conclusions do not change when new facts are added.
    - **Example:** A fixed conclusion in a legal case, even if more evidence is discovered.
6. **Non-monotonic Reasoning:**
    - **Description:** Conclusions can change when new information is introduced, making it flexible.
    - **Example:** If you conclude "John is at the office" but later find out he’s at home, the conclusion changes.
7. **Forward & Backward Reasoning:**
    - **Forward Reasoning:** Starts with known facts and applies rules to reach a conclusion (used in expert systems).
    - **Backward Reasoning:** Starts with a goal and works backward to find supporting facts (used in diagnostic systems).

Expert System Shells: An **Expert System Shell** is a software framework that helps users create expert systems by adding their domain-specific knowledge. It provides essential components like the knowledge base and reasoning engine, making it easier to build and customize expert systems.

Key Components of an Expert System Shell:

1. **Knowledge Base:**
    - Stores factual and heuristic knowledge about a domain, using rules, facts, and relationships.
    - Example: A medical system might have rules about symptoms and diseases.
2. **Reasoning Engine:**
    - Manipulates the knowledge base and derives conclusions using inference mechanisms like backward chaining or case-based reasoning.
    - Example: In troubleshooting, it may use backward chaining to identify causes of issues.
3. **Knowledge Acquisition Subsystem:**
    - Helps collect and structure knowledge from domain experts to build the knowledge base.
    - Challenge: Gathering expert knowledge can be difficult and time-consuming.
4. **Explanation Subsystem:**
    - Explains the reasoning behind the system's decisions or diagnoses, showing how it arrived at a conclusion.
    - Example: In medical diagnosis, it may explain how symptoms led to a particular disease diagnosis.
5. **User Interface:**
    - Allows users to interact with the expert system, input queries, and view solutions.
    - Example: A GUI that allows users to input symptoms and get a diagnosis.

Benefits of Expert System Shells:

- **Flexibility:** Users can easily add new knowledge.
- **Efficiency:** Provides a ready-to-use framework to speed up system development.
- **Customization:** Users can tailor the system for specific domains.

Examples of Expert System Shells:

- **EMYCIN:** A simplified version of MYCIN, used for building medical diagnostic systems, employing backward chaining for reasoning.
- **CLIPS:** A versatile rule-based shell developed by NASA, supporting both forward and backward chaining for various domains.
- **JESS:** A Java-based shell that integrates with Java applications, often used in business rule engines and decision support systems.

Knowledge acquisition is the process of gathering, organizing, and analyzing information to improve efficiency and decision-making. Here are the key steps:

1. Data Gathering: Collecting relevant data from various sources.

- **Objective:** Ensure the data is accurate and relevant for future use.
- **Example:** Gathering customer feedback, sales data, and market trends.

2. Data Organizing: Structuring raw data to make it usable.

- **Objective:** Transform unorganized data into a more accessible format.
- **Example:** Categorizing customer data by region, age group, or product type.

3. Summarizing: Processing raw data into easy-to-understand statistics and visuals.

- **Objective:** Convert complex data into useful information.
- **Example:** Presenting sales data in tables, graphs, or charts to highlight trends.

4. Analyzing: Examining data for patterns or trends that could lead to new knowledge.

- **Objective:** Identify insights that could improve decision-making.
- **Example:** Finding that certain products sell better during specific seasons or in certain locations.

5. Synthesizing: Combining patterns and trends into reliable knowledge, typically stored in a knowledge base.

- **Objective:** Create usable rules or guidelines for future decision-making.
- **Example:** Establishing a rule that certain products tend to be purchased together, influencing inventory and marketing strategies.

Example: Retail Store Company(Through this process, the company acquires valuable knowledge that improves operations, optimizes inventory, and informs marketing strategies.)

1. **Data Collection:**
    - A retail company collects data from store locations, customer purchases, and external factors like weather and competitor actions.
2. **Data Organizing:**
    - The data is organized into structured databases by product categories, customer demographics, and external influences.
3. **Summarizing:**
    - Reports and graphs are created to summarize product sales and customer preferences by demographic group.
4. **Analyzing:**
    - Patterns emerge, such as customers who buy nappies often also purchase wet towels, and store locations with varying product sales.
5. **Synthesizing:**
    - Knowledge is created from patterns, such as the relationship between nappies and wet towels, which can guide marketing and product placement.